Lesson 16：多轮聊天和上下文会话管理

LLM Learn 大模型开发课程

2026 年 11 月

目录

1 教学目标与先修 2

2 状态从哪里来 2

3 Conversation 类的整体逻辑 2

4 交互命令与隐私 2

5 测试用例 2

6 核心关系或公式 2

7 本课完整源码：逐行中文注释 2

8 练习与验收 4

9 执行记录与参考 5

1 教学目标与先修

理解 response ID 的作用、状态保存与重置，做出可测试的多轮 CLI 助手。

先修：Lesson 15 单轮 Responses API。预计 90–120 分钟。

2 状态从哪里来

应用需要保存上一轮 Response ID,并在下一轮请求里指定 previous_response_id;它不是 Python 自动记忆。store=True 用于服务端关联会话，历史上下文依然可能产生 Token 费用。

3 Conversation 类的整体逻辑

___init___ 保存 client、model 和 previous_response_id；ask 构造请求参数并仅在调用成功时更新 ID；reset 清除本地 ID，使下一轮重新开始。失败时保留上一轮合法状态。

4 交互命令与隐私

/exit 结束 CLI；/reset 只清除本地关联，不代表删除服务端已保存的数据。请勿用教学程序提交真实客户数据或个人秘密。

5 测试用例

FakeResponses 为每次 create 提供递增 ID;断言第二轮发送 fake-1,重置后没有 previous_response_id;空白输入不应产生请求。

6 核心关系或公式

$$s_{t+1} = \mathrm{id}(\mathrm{Response}_t)$$

7 本课完整源码：逐行中文注释

下列程序与本课 code 目录的实际源文件完全对应。每个模块、函数及关键可执行语句都配有中文说明。

```
"""Lesson 16: 基于 Responses API 的多轮聊天；真实调用需要 Key 和额度。"""

# 导入 os，读取 API Key 及可覆盖的模型配置。
import os
# 导入 OpenAI SDK 的客户端与错误基类。
from openai import OpenAI, OpenAIError
```

```
# 默认教学模型可以根据模型权限调整。
MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
# 稳定的指令提示用于约束教程回答风格。
SYSTEM_INSTRUCTIONS = " 你是严谨的 Python 和大模型开发课程教师；遇到不确定内容明确说明。"


class Conversation:
    """ 维护 Responses API 会话 ID，并提供提问与重置操作。"""

    def __init__(self, client, model: str = MODEL) -> None:
        """client 为真实或模拟 SDK；model 为有权限使用的模型名称。"""
        # 保存客户端对象，使其能够在测试中替换成 FakeClient。
        self.client = client
        # 保存模型名，避免每次提问反复传参。
        self.model = model
        # None 表示当前没有上一轮可衔接的响应。
        self.previous_response_id = None

    def reset(self) -> None:
        """ 清除本地上一轮响应 ID，使下次提问开启新对话。"""
        # 把关联响应重置为空；不会自动删除服务端保留的数据。
        self.previous_response_id = None

    def ask(self, question: str) -> str:
        """ 发送新问题并链接上一轮响应；失败时保留旧会话状态。"""
        # 空问题不应该触发付费模型请求。
        if not question.strip():
            # 使用清晰的异常提示帮助调用者处理输入错误。
            raise ValueError(" 问题不能为空")
        # 构建 SDK 所需的命名参数字典。
        params = {
            # 指定当前使用的 LLM 模型。
            "model": self.model,
            # 每轮显式加入稳定指令，避免在续聊中丢失约束。
            "instructions": SYSTEM_INSTRUCTIONS,
            # 用户本轮提交的文本。
            "input": question,
            # 允许 API 保留本次响应，用于下轮 response id 续聊。
            "store": True,
            # 控制最长生成长度，减少教学实验的不必要花费。
            "max_output_tokens": 500,
        }
        # 只有完成至少一轮后，才需要传入上一次响应 ID。
        if self.previous_response_id is not None:
            # SDK 通过该字段把当前请求与历史响应关联起来。
            params["previous_response_id"] = self.previous_response_id
        # 真正的网络调用发生在 create；测试时可完全用 mock 替换。
        response = self.client.responses.create(**params)
        # 仅在调用成功后更新 ID，避免失败导致会话被破坏。
        self.previous_response_id = response.id
        # 输出由 SDK 汇总的文本回复。
        return response.output_text


def main() -> None:
    """ 运行交互式命令行聊天，支持 /reset 与 /exit。"""
    # 没有环境变量时避免实例化真实 API 客户端。
```

```
    if not os.getenv("OPENAI_API_KEY"):
        # 输出修复步骤，绝不打印任何密钥内容。
        print(" 未发现 OPENAI_API_KEY：请先配置 API 密钥。")
        # 直接结束，避免错误的在线调用记录。
        return
    # 初始化具有超时和有限重试的 SDK 客户端。
    client = OpenAI(timeout=45.0, max_retries=1)
    # 创建对话控制对象，集中管理续聊状态。
    session = Conversation(client=client)
    # 告知用户退出和清除本地会话链接的命令。
    print(" 多轮聊天已启动；输入 /reset 开始新会话，输入 /exit 退出。")
    # 使用循环持续接受用户文本问题。
    while True:
        try:
            # 从标准输入读取问题，并去掉首尾空白。
            question = input("\n 你> ").strip()
        except (EOFError, KeyboardInterrupt):
            # Ctrl+C 或输入结束时友好退出。
            print("\n 会话已结束")
            # 离开循环，避免反复读取失败的输入。
            break
        # 忽略空输入，不产生额外 API 请求。
        if not question:
            # 返回循环开头，等待下一条有效输入。
            continue
        # 处理用户输入的退出命令。
        if question == "/exit":
            # 结束交互会话循环。
            break
        # 处理用户输入的重置命令。
        if question == "/reset":
            # 清空上一轮响应 ID，但不触碰服务器端数据保留策略。
            session.reset()
            # 向用户解释本地会话关联已经重置。
            print(" 本地对话已重置。")
            # 继续下一轮输入，不发送模型请求。
            continue
        try:
            # 发送用户新问题并读取模型返回的文本。
            answer = session.ask(question)
        except (OpenAIError, ValueError) as error:
            # 网络、权限、配额或输入错误都应明确显示失败。
            print(" 本轮调用失败:", error)
            # 失败不清空会话 ID，下轮仍可使用已成功的上次响应。
            continue
        # 把模型回答打印到控制台。
        print(" 助手>", answer)


if __name__ == "__main__":
    # 确保从别的模块导入 Conversation 时不会启动聊天循环。
    main()
```

8 练习与验收

- 连续两轮保持合法 previous_response_id。

- 调用 reset 后恢复新对话。
- 模拟测试通过，但不得声称已真实在线问答。

9 执行记录与参考

使用 python -m unittest discover -s tests -v 执行测试。请以本课 results/verification.md 中的真实结果和 GitHub Actions 日志为准，Mock 并非在线 API 验证。

- https://git-scm.com/docs
- https://docs.github.com/en/actions
- https://developer.mozilla.org/en-US/docs/Web/HTTP
- https://platform.openai.com/docs/api-reference/responses